Quality Assurance Concept for Chess Mania

Quality Assurance is a broad term, explained on the Google Testing Blog as "*the continuous and consistent improvement and maintenance of process that enables the QC job*". As follows from the definition, QA focuses more on organisational aspects of quality management and monitoring the consistency of the production process.

There are a wide variety of 'best practices' for QA currently adopted in software development, we have decided to implement the following procedures in working on our project:

Communication:

Our Team uses Zoom to attend meetings where the very next logical development steps are decided and tasks are assigned to team members. The Gantt Project file with all the milestone achievements is used as a compass to guide this decision. After assignment a Trello chart is created where each member can self-assign a task that he/she is working on. This is to ensure transparency in the working process and avoid unpleasant surprises.

If possible we work with the Zoom meeting online so team members can ask for help if needed. If a synchronous session is not an option we use Slack for ongoing communication about tasks and issues and for sharing relevant resources.

After a Milestone is reached, the process begins anew.

Explaining the Code / Code Review:

After a task is completed, we regularly explain our implementation to each other. This is to ensure that there are no obvious mistakes left in the code and also to ensure that team members are up to date on the status of the project. This process also doubles as a Code Review where the work is checked for style (we are using the Google Style Guide) and content.

Documenting the Code:

All code is extensively documented in English using Javadoc for later reference.

Exception Handling / Logging:

All relevant methods use a try-catch construct to handle exceptions. We agreed on using the logging framework log4j2 to log errors in the hope that this will allow more constructive error handling.

Testing

We test our code (Black Box & White Box) as soon as possible to make sure it functions as expected so that any detected issues are fixed and do not snowball. This is crucial because the cost of an error tends to increase throughout the software development process.

Unit Tests

We will use JUnit - Jacoco to test a central component of our game as required by the Milestone Checklist.

Metrics

We agreed to use the IntelliJ Metrics Reloaded Plugin to monitor the quantitative features of our source code.